

LAPORAN TUGAS BESAR

Implementasi Algoritma Greedy pada Robocode Tank Royale

[Sisipkan foto bersama ketiga anggota kelompok di sini]

Ukuran disarankan: 10 cm x 7 cm

Disusun oleh:

Bayu Aditya Tarigan (124140065)

Ganda Tua Sitio (124140143)

Rafael Alfredo Ginting (124140220)

INSTITUT TEKNOLOGI SUMATERA

Program Studi Teknik Informatika

IF2211 - Strategi Algoritma

Semester Genap 2026

DAFTAR ISI

BAB 1: DESKRIPSI TUGAS.....	4
1.1 Latar Belakang	4
1.2 Tujuan Tugas	4
1.3 Batasan Masalah	5
1.4 Sistematika Laporan.....	5
BAB 2: LANDASAN TEORI	6
2.1 Algoritma Greedy	6
2.1.1 Pengertian Algoritma Greedy	6
2.1.2 Komponen Algoritma Greedy.....	6
2.1.3 Kelebihan dan Kekurangan Algoritma Greedy.....	6
2.2 Robocode Tank Royale.....	7
2.2.1 Pengenalan Robocode Tank Royale	7
2.2.2 Cara Kerja Program Bot.....	7
2.2.3 Implementasi Algoritma Greedy pada Bot	7
2.2.4 Cara Menjalankan Bot	8
BAB 3: APLIKASI STRATEGI GREEDY	9
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy	9
3.2 Eksplorasi 4 Alternatif Solusi Greedy	9
3.2.1 Alternatif 1: Greedy by Distance	9
3.2.2 Alternatif 2: Greedy by Hit Probability	10
3.2.3 Alternatif 3: Greedy by Safe Movement.....	10
3.2.4 Alternatif 4: Greedy by Chaos Movement.....	11
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi.....	11
3.4 Strategi Greedy yang Dipilih	12
BAB 4: IMPLEMENTASI DAN PENGUJIAN.....	13
4.1 Implementasi Bot Utama Yang Digunakan	13
4.1.1 Pseudocode Bot Utama (BotKeempat)	13
4.2.2 Pseudocode Bot Alternatif 2 (BotPertama).....	14
4.2.3 Pseudocode Bot Alternatif 3 (BotKetiga)	15
4.2.4 Pseudocode Bot Alternatif 4 (BotKedua)	15
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot Terpilih	16
4.2.1 Struktur Data yang Digunakan.....	17
4.2.2 Fungsi-Fungsi Utama.....	17
4.2.3 Prosedur Event Handler	18
4.3 Pengujian.....	18

4.3.1 Pengujian 1: Kondisi Normal pada Arena Standar	18
4.3.2 Pengujian 2: Melawan Bot Agresif	19
4.3.3 Pengujian 3: Melawan Bot Defensif	20
4.4 Analisis Hasil Pengujian	20
BAB 5: KESIMPULAN DAN SARAN	22
5.1 Kesimpulan	22
5.2 Saran	22
LAMPIRAN.....	24
Lampiran A: Tautan Repository GitHub	24
Lampiran B: Tautan Video Demo (Opsional)	24
DAFTAR PUSTAKA	25

BAB 1: DESKRIPSI TUGAS

1.1 Latar Belakang

Perkembangan teknologi di bidang kecerdasan buatan membuat banyak permainan komputer mulai memanfaatkan algoritma untuk mengatur perilaku karakter atau agen otomatis. Salah satu algoritma yang cukup sering digunakan adalah algoritma greedy. Algoritma ini bekerja dengan cara memilih keputusan terbaik pada setiap langkah berdasarkan kondisi yang sedang terjadi saat itu.

Pada tugas besar mata kuliah Strategi Algoritma ini, algoritma greedy diterapkan pada permainan Robocode Tank Royale. Robocode Tank Royale merupakan permainan pertarungan tank virtual berbasis pemrograman, di mana pemain harus membuat bot yang dapat bergerak, menyerang, bertahan, dan mengambil keputusan secara otomatis selama pertandingan berlangsung.

Setiap bot harus memiliki strategi agar dapat memperoleh skor setinggi mungkin. Skor dalam permainan diperoleh dari beberapa aspek, seperti damage yang diberikan kepada musuh, kemampuan bertahan hidup, dan keberhasilan mengalahkan lawan. Oleh karena itu, algoritma greedy dipilih karena mampu mengambil keputusan dengan cepat dan cocok digunakan pada permainan yang berjalan secara real-time.

Dalam tugas besar ini dibuat empat strategi greedy yang berbeda. Masing-masing strategi menggunakan heuristic yang berbeda dalam menentukan target maupun tindakan bot. Selanjutnya, seluruh strategi dibandingkan untuk menentukan strategi terbaik yang akan digunakan sebagai bot utama.

Bot utama yang digunakan dalam tugas besar ini adalah BotKeempat. BotKeempat menggunakan strategi greedy berbasis chaos movement, yaitu bot bergerak secara acak dengan pola zigzag dan perubahan arah mendadak untuk mempersulit lawan memprediksi posisi bot. Strategi ini dipilih karena memperoleh total score tertinggi dan memenangkan seluruh ronde pertandingan dibandingkan bot lainnya.

Melalui tugas besar ini diharapkan mahasiswa dapat memahami penerapan algoritma greedy dalam menyelesaikan permasalahan nyata, khususnya pada pengembangan bot permainan otomatis.

1.2 Tujuan Tugas

Tujuan dari tugas besar ini adalah:

- Memahami konsep dasar algoritma greedy.
- Mengimplementasikan algoritma greedy pada permainan Robocode Tank Royale.
- Membuat empat strategi greedy dengan heuristic yang berbeda-beda.
- Menganalisis efektivitas strategi greedy dalam pertandingan.
- Mengembangkan bot yang mampu memperoleh skor tinggi dalam permainan.

1.3 Batasan Masalah

Batasan masalah dalam tugas besar ini adalah sebagai berikut:

- Bot dibuat menggunakan bahasa pemrograman C# (.NET 6.0).
- Permainan menggunakan game engine Robocode Tank Royale yang telah dimodifikasi oleh asisten.
- Strategi yang digunakan hanya berbasis algoritma greedy.
- Pengujian dilakukan pada mode pertandingan 1 vs 1.
- Bot hanya memanfaatkan informasi yang diperoleh dari radar dan event permainan.
- Setiap strategi greedy harus memiliki heuristic yang berbeda.

1.4 Sistematika Laporan

Laporan ini disusun dengan sistematika sebagai berikut:

1. Bab 1 membahas deskripsi tugas.
2. Bab 2 membahas landasan teori.
3. Bab 3 membahas aplikasi strategi greedy.
4. Bab 4 membahas implementasi dan pengujian.
5. Bab 5 membahas kesimpulan dan saran.

BAB 2: LANDASAN TEORI

2.1 Algoritma Greedy

2.1.1 Pengertian Algoritma Greedy

Algoritma greedy merupakan salah satu metode penyelesaian masalah yang dilakukan dengan cara memilih keputusan terbaik pada setiap langkah berdasarkan kondisi yang sedang dihadapi saat itu. Pendekatan greedy tidak mempertimbangkan seluruh kemungkinan yang akan terjadi di masa depan, tetapi hanya fokus pada pilihan yang dianggap paling menguntungkan pada langkah sekarang.

Tujuan utama dari algoritma greedy adalah memperoleh solusi yang optimal atau mendekati optimal dengan proses yang lebih cepat dan sederhana dibandingkan metode lainnya. Karena proses pengambilan keputusan dilakukan secara langsung pada setiap langkah, algoritma ini cocok digunakan pada sistem yang membutuhkan respon cepat, salah satunya pada permainan Robocode Tank Royale.

Pada permainan Robocode Tank Royale, bot harus mampu mengambil keputusan dalam waktu yang sangat singkat pada setiap turn. Oleh karena itu, penggunaan algoritma greedy dinilai sesuai karena bot dapat langsung menentukan tindakan terbaik seperti memilih target, menentukan arah gerakan, maupun menentukan waktu menyerang berdasarkan kondisi arena saat itu.

2.1.2 Komponen Algoritma Greedy

Algoritma greedy memiliki komponen-komponen utama berikut:

6. Himpunan kandidat: Sekumpulan elemen yang dapat dipilih untuk membentuk solusi.
7. Himpunan solusi: Kumpulan kandidat yang telah dipilih menjadi solusi sementara atau solusi akhir.
8. Fungsi seleksi: Fungsi yang menentukan kandidat terbaik yang dipilih pada setiap langkah.
9. Fungsi kelayakan: Fungsi yang memeriksa apakah kandidat yang dipilih memenuhi syarat untuk dimasukkan ke solusi.
10. Fungsi objektif: Fungsi yang digunakan untuk mengukur kualitas solusi, misalnya memaksimalkan damage atau meminimalkan risiko.

2.1.3 Kelebihan dan Kekurangan Algoritma Greedy

Algoritma greedy memiliki beberapa kelebihan, di antaranya mudah dipahami dan diimplementasikan. Selain itu, proses eksekusinya relatif cepat karena hanya memilih solusi terbaik pada setiap langkah tanpa harus memeriksa seluruh kemungkinan solusi yang ada. Hal ini membuat algoritma greedy cocok digunakan pada sistem real-time seperti Robocode Tank Royale yang memiliki batas waktu pada setiap turn.

Meskipun demikian, algoritma greedy juga memiliki kekurangan. Algoritma ini tidak selalu menghasilkan solusi global yang paling optimal karena keputusan yang diambil hanya berdasarkan kondisi saat itu. Pada beberapa kondisi tertentu, pilihan yang terlihat paling baik di awal belum tentu menghasilkan hasil terbaik pada akhir pertandingan. Selain itu, performa algoritma greedy sangat bergantung pada heuristic yang digunakan.

2.2 Robocode Tank Royale

2.2.1 Pengenalan Robocode Tank Royale

Robocode Tank Royale adalah permainan simulasi pertarungan tank berbasis pemrograman. Pada permainan ini, pemain membuat bot otomatis menggunakan bahasa pemrograman tertentu untuk bertarung di dalam arena. Setiap bot memiliki kemampuan untuk bergerak di arena, memutar radar dan meriam, mendeteksi musuh, menembak musuh, dan menghindari serangan lawan. Pemenang pertandingan ditentukan berdasarkan skor dan kemampuan bertahan hidup bot.

2.2.2 Cara Kerja Program Bot

Bot bekerja berdasarkan loop utama yang dijalankan terus-menerus selama pertandingan berlangsung. Pada setiap turn, bot akan memindai arena menggunakan radar, mengumpulkan informasi musuh, memilih aksi berdasarkan strategi tertentu, lalu menjalankan aksi seperti bergerak atau menembak. Robocode menyediakan berbagai event handler seperti OnScannedBot, OnHitWall, OnHitBot, dan OnBotDeath yang digunakan agar bot dapat bereaksi terhadap kondisi permainan secara real-time.

2.2.3 Implementasi Algoritma Greedy pada Bot

Implementasi greedy dilakukan dengan memilih aksi terbaik pada setiap turn berdasarkan heuristic tertentu. Contohnya: memilih musuh terdekat, memilih musuh dengan energi terendah, memprediksi posisi musuh saat peluru tiba, atau memilih gerakan yang paling sulit ditebak lawan. Bot tidak menghitung semua kemungkinan masa depan, melainkan langsung mengambil keputusan yang dianggap paling menguntungkan saat itu.

2.2.4 Cara Menjalankan Bot

Langkah menjalankan bot pada Robocode Tank Royale:

11. Install Java Development Kit (JDK) versi 11 atau lebih baru.
12. Install .NET SDK 6.0.
13. Jalankan server GUI Robocode Tank Royale: `java -jar robocode-tankroyale-gui-0.30.0.jar`
14. Masuk ke folder bot dan jalankan: `dotnet run`
15. Pilih bot pada GUI dan mulai pertandingan.

BAB 3: APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy

Pada permainan Robocode Tank Royale, bot harus mampu mengambil keputusan dengan cepat di setiap turn agar dapat memperoleh skor setinggi mungkin. Permasalahan ini dapat dimodelkan ke dalam elemen-elemen algoritma greedy karena bot harus selalu memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi saat itu juga.

Elemen Greedy	Konteks Robocode Tank Royale
Himpunan Kandidat	Semua musuh yang berhasil dideteksi radar bot selama pertandingan berlangsung.
Himpunan Solusi	Kumpulan aksi yang dilakukan bot seperti bergerak, mengejar musuh, menghindari, menembak, dan menabrak lawan.
Fungsi Solusi	Kondisi ketika bot berhasil memperoleh skor tinggi atau memenangkan pertandingan dengan mengalahkan lawan.
Fungsi Seleksi	Proses memilih target atau aksi terbaik berdasarkan heuristic tertentu, misalnya memilih musuh terdekat atau musuh yang paling mudah diprediksi posisinya.
Fungsi Kelayakan	Memastikan aksi yang dipilih dapat dilakukan, misalnya posisi musuh masih dalam jangkauan radar atau meriam tidak dalam kondisi panas.
Fungsi Objektif	Memaksimalkan total skor pertandingan melalui bullet damage, survival score, ram damage, dan bonus kemenangan.

Pada implementasinya, bot akan terus melakukan scanning terhadap arena. Setelah musuh ditemukan, bot akan menentukan tindakan terbaik berdasarkan strategi greedy yang digunakan. Keputusan tersebut dilakukan secara berulang pada setiap turn sehingga bot dapat terus beradaptasi terhadap kondisi pertandingan.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

Dalam tugas besar ini dibuat empat alternatif strategi greedy yang berbeda. Masing-masing strategi menggunakan heuristic yang berbeda dalam menentukan target maupun tindakan bot selama pertandingan berlangsung. Tujuan dari eksplorasi ini adalah mencari strategi yang paling efektif untuk memperoleh skor tertinggi pada permainan Robocode Tank Royale.

3.2.1 Alternatif 1: Greedy by Distance

Strategi ini diterapkan pada BotPertama. Bot akan selalu memilih musuh yang memiliki jarak paling dekat lalu langsung bergerak mendekati target tersebut untuk melakukan serangan. Strategi ini termasuk algoritma greedy karena pada setiap putaran bot

selalu mengambil keputusan lokal terbaik yaitu menyerang musuh yang paling mudah dijangkau tanpa mempertimbangkan kondisi jangka panjang.

Heuristic yang digunakan:

- Memilih musuh dengan jarak paling dekat sebagai target prioritas.
- Menyesuaikan fire power berdasarkan jarak: jarak < 200 unit = power 3, jarak < 400 unit = power 2, jarak lebih jauh = power 1.
- Bergerak langsung menuju musuh sejauh setengah jarak untuk meminimalkan waktu pendekatan.

3.2.2 Alternatif 2: Greedy by Hit Probability

Strategi ini diterapkan pada BotKeempat. Bot akan memilih target dengan peluang terkena tembakan paling tinggi berdasarkan prediksi posisi musuh. BotKeempat menghitung posisi musuh saat peluru tiba menggunakan rumus linear prediction dan menembak ke posisi prediksi tersebut. Strategi ini termasuk greedy karena bot selalu memilih peluang hit terbaik pada setiap putaran permainan.

Heuristic yang digunakan:

- Memprediksi posisi musuh saat peluru tiba menggunakan rumus: $\text{predictedX} = eX + \sin(\text{heading}) \times \text{speed} \times \text{travelTime}$.
- Gun dan radar independen dari badan sehingga bot dapat berputar bebas tanpa mempengaruhi arah tembakan.
- Radar oscillation agar musuh tetap terlacak secara berkelanjutan.
- Menembak hanya jika sudut gun terhadap posisi prediksi kurang dari 5 derajat untuk memastikan akurasi.

3.2.3 Alternatif 3: Greedy by Safe Movement

Strategi ini digunakan pada BotKetiga. Fokus utama bot adalah bergerak terus-menerus di sepanjang tepi arena untuk meminimalkan sudut serangan dari musuh sambil tetap menyerang ketika ada kesempatan. Strategi ini termasuk greedy karena bot selalu memilih gerakan yang dianggap paling aman pada kondisi saat itu.

Heuristic yang digunakan:

- Bergerak terus-menerus di sepanjang dinding arena agar selalu mobile dan sulit dijadikan target.

- Senjata independen dari badan sehingga dapat menyerang dari berbagai sudut tanpa berhenti bergerak.
- Fire power disesuaikan: jarak < 300 = power 3, jarak < 500 = power 2, jarak lebih jauh = power 1.
- Membalik arah gerak saat menabrak dinding untuk tetap bergerak tanpa tersangkut di sudut arena.

3.2.4 Alternatif 4: Greedy by Chaos Movement

Strategi ini digunakan pada BotKeempat. Fokus utama bot adalah membuat gerakannya tidak terprediksi melalui zigzag acak sehingga sangat sulit ditembak, sambil menembak secara berkelanjutan ke semua arah. Strategi ini termasuk greedy karena bot selalu memilih gerakan paling acak yang meminimalkan peluang lawan menebak posisinya.

Heuristic yang digunakan:

- Gerakan zigzag dengan sudut acak antara 30 hingga 90 derajat dan jarak maju antara 50 hingga 150 unit setiap langkah.
- Menembak secara berkelanjutan setiap kali radar selesai berputar penuh.
- Membalik arah gerak secara langsung saat mendeteksi musuh atau kena tembakan untuk mempersulit prediksi posisi.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Keempat strategi greedy memiliki karakteristik yang berbeda-beda. Perbandingan dilakukan berdasarkan kompleksitas algoritma, efektivitas saat pertandingan, serta kelebihan dan kekurangan masing-masing strategi.

Alternatif	Kompleksitas	Efektivitas	Kelebihan	Kekurangan
Greedy by Distance (BotPertama)	$O(1)$	Tinggi	Agresif, sederhana, efektif untuk pertarungan jarak dekat	Rentan terkena serangan balik saat mendekati musuh
Greedy by Hit Probability (BotKeempat)	$O(1)$	Sangat Tinggi	Akurasi tembakan tertinggi, prediksi posisi musuh secara linear	Kurang efektif bila musuh bergerak sangat tidak menentu
Greedy by Safe Movement (BotKetiga)	$O(1)$	Sedang	Selalu bergerak, susah ditembak, hemat energi	Kurang agresif, total damage lebih rendah dibanding bot lain

Alternatif	Kompleksitas	Efektivitas	Kelebihan	Kekurangan
Greedy by Chaos Movement (BotKeempat)	$O(1)$	Sedang	Gerakan tidak terprediksi, sangat sulit dijadikan target	Boros energi karena menembak terus, akurasi tembakan rendah

Keterangan: n merupakan jumlah musuh yang berhasil dipindai radar. Semua strategi menggunakan pendekatan greedy karena keputusan diambil berdasarkan kondisi terbaik pada saat itu tanpa mempertimbangkan seluruh kemungkinan di masa depan.

3.4 Strategi Greedy yang Dipilih

Berdasarkan hasil analisis terhadap empat alternatif strategi greedy yang telah dibuat, strategi yang dipilih untuk diimplementasikan pada bot utama adalah Greedy by Hit Probability yang diterapkan pada BotKeempat. Strategi ini dipilih karena memiliki performa terbaik berdasarkan hasil pertandingan dan memperoleh total score tertinggi di antara seluruh bot.

Pada strategi ini, bot menggunakan gerakan chaos berupa zigzag acak dan perubahan arah mendadak untuk mempersulit lawan dalam memprediksi posisi bot. Selain itu, bot terus melakukan tembakan agresif selama pertandingan berlangsung untuk memberikan tekanan kepada lawan.

Dibandingkan alternatif lainnya, BotKeempat unggul dalam kemampuan bertahan hidup, mobilitas, dan konsistensi memperoleh score tinggi. Gerakan yang sulit diprediksi membuat lawan kesulitan mengenai target, sehingga BotKeempat mampu bertahan lebih lama dan menghasilkan damage terbesar selama pertandingan.

Oleh karena itu, strategi Greedy by Chaos Movement dipilih sebagai strategi utama pada BotKeempat karena terbukti paling efektif berdasarkan hasil pertandingan dengan total score tertinggi dan kemenangan terbanyak.

BAB 4: IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Bot Utama Yang Digunakan

4.1.1 Pseudocode Bot Utama (BotKeempat)

```
using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

public class BotKeempat : Bot
{
    static void Main(string[] args) => new BotKeempat().Start();
    BotKeempat() : base(BotInfo.FromFile("BotKeempat.json")) { }

    int turnDir = 1;
    Random rng = new Random();

    public override void Run()
    {
        BodyColor = Color.DarkMagenta;
        GunColor = Color.HotPink;
        BulletColor = Color.White;

        while (IsRunning)
        {
            TurnRight(rng.Next(30, 90) * turnDir);
            Forward(rng.Next(50, 150));
            TurnGunRight(360);
            Fire(1);
        }
    }

    public override void OnScannedBot(ScannedBotEvent e)
    {
        double d = DistanceTo(e.X, e.Y);
        Fire(d < 150 ? 3 : 2);
        turnDir *= -1;
        TurnRight(45 * turnDir);
    }

    public override void OnHitByBullet(HitByBulletEvent e)
    {
        turnDir *= -1;
        TurnRight(60 * turnDir);
        Forward(100);
    }

    public override void OnHitWall(HitWallEvent e)
    {
        Back(50);
        TurnRight(rng.Next(45, 135));
    }
}
```

4.2.2 Pseudocode Bot Alternatif 2 (BotPertama)

```
using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

public class BotPertama : Bot
{
    static void Main(string[] args) => new BotPertama().Start();
    BotPertama() : base(BotInfo.FromFile("BotPertama.json")) { }

    double enemyX, enemyY, enemySpeed, enemyHeading;
    bool enemySeen = false;

    public override void Run()
    {
        BodyColor = Color.DarkRed;
        GunColor = Color.OrangeRed;
        BulletColor = Color.Yellow;

        while (IsRunning)
        {
            TurnGunRight(360); // terus scan
        }
    }

    public override void OnScannedBot(ScannedBotEvent e)
    {
        enemyX = e.X;
        enemyY = e.Y;
        enemySpeed = e.Speed;
        enemyHeading = e.Direction;
        enemySeen = true;

        // Hitung sudut ke musuh
        double angleToEnemy = BearingTo(e.X, e.Y);
        TurnLeft(angleToEnemy); // badan langsung ke musuh
        Forward(DistanceTo(e.X, e.Y) * 0.5); // kejar setengah jarak

        // Tembak sekencang mungkin
        double distance = DistanceTo(e.X, e.Y);
        double firePower = distance < 200 ? 3 : distance < 400 ? 2 : 1;
        Fire(firePower);
    }

    public override void OnHitBot(HitBotEvent e)
    {
        // Nabrak musuh = tembak maksimal
        Fire(3);
        Back(50);
        TurnRight(30);
    }

    public override void OnHitWall(HitWallEvent e)
    {
        Back(40);
        TurnRight(45);
    }

    public override void OnHitByBullet(HitByBulletEvent e)
    {
        // Dodge ke kanan saat kena tembak
        TurnRight(45);
        Forward(60);
    }
}
```

4.2.3 Pseudocode Bot Alternatif 3 (BotKetiga)

```
1 using System;
2 using System.Drawing;
3 using Robocode.TankRoyale.BotApi;
4 using Robocode.TankRoyale.BotApi.Events;
5
6 public class BotKetiga : Bot
7 {
8     static void Main(string[] args) => new BotKetiga().Start();
9     BotKetiga() : base(BotInfo.FromFile("BotKetiga.json")) { }
10
11     int moveDir = 1;
12
13     public override void Run()
14     {
15         BodyColor = Color.Navy;
16         GunColor = Color.Cyan;
17
18         while (IsRunning)
19         {
20             Forward(100 * moveDir);
21             TurnGunRight(360);
22         }
23     }
24
25     public override void OnScannedBot(ScannedBotEvent e)
26     {
27         double distance = DistanceTo(e.X, e.Y);
28         double power = distance < 300 ? 3 : distance < 500 ? 2 : 1;
29
30         double gunTurn = GunBearingTo(e.X, e.Y);
31         TurnGunRight(gunTurn);
32         Fire(power);
33     }
34
35     public override void OnHitWall(HitWallEvent e)
36     {
37         moveDir *= -1;
38         Back(20);
39         TurnRight(90);
40     }
41
42     public override void OnHitBot(HitBotEvent e)
43     {
44         Fire(3);
45         Back(50);
46         moveDir *= -1;
47     }
48
49     public override void OnHitByBullet(HitByBulletEvent e)
50     {
51         TurnRight(30 * moveDir);
52         moveDir *= -1;
53     }
54 }
55 }
```

4.2.4 Pseudocode Bot Alternatif 4 (BotKedua)

```

1  using System;
2  using System.Drawing;
3  using Robocode.TankRoyale.BotApi;
4  using Robocode.TankRoyale.BotApi.Events;
5
6  public class BotKedua : Bot
7  {
8      static void Main(string[] args) => new BotKedua().Start();
9      BotKedua() : base(BotInfo.FromFile("BotKedua.json")) { }
10
11     public override void Run()
12     {
13         BodyColor = Color.Black;
14         GunColor = Color.Lime;
15         RadarColor = Color.Lime;
16         BulletColor = Color.Lime;
17
18         while (IsRunning)
19         {
20             TurnRadarRight(360);
21         }
22     }
23
24     public override void OnScannedBot(ScannedBotEvent e)
25     {
26         double firePower = 3.0;
27         double bulletSpeed = 20 - 3 * firePower;
28
29         double distance = DistanceTo(e.X, e.Y);
30         double travelTime = distance / bulletSpeed;
31
32         double enemyHeadingRad = e.Direction * Math.PI / 180.0;
33         double predictedX = e.X + Math.Sin(enemyHeadingRad) * e.Speed * travelTime;
34         double predictedY = e.Y + Math.Cos(enemyHeadingRad) * e.Speed * travelTime;
35
36         predictedX = Math.Max(18, Math.Min(ArenaWidth - 18, predictedX));
37         predictedY = Math.Max(18, Math.Min(ArenaHeight - 18, predictedY));
38
39         double gunAngle = GunBearingTo(predictedX, predictedY);
40         TurnGunRight(gunAngle);
41
42         if (Math.Abs(gunAngle) < 5)
43             Fire(firePower);
44
45         double radarAngle = RadarBearingTo(e.X, e.Y);
46         TurnRadarRight(radarAngle * 2);
47     }
48
49     public override void OnHitWall(HitWallEvent e)
50     {
51         Back(30);
52         TurnRight(90);
53     }
54
55     public override void OnHitByBullet(HitByBulletEvent e)
56     {
57         TurnRight(60);
58         Forward(80);
59     }
60 }

```

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot Terpilih

BotKeempat menggunakan beberapa variabel sederhana seperti turnDir untuk menentukan arah gerakan dan Random untuk menghasilkan pergerakan acak. Fungsi utama pada bot ini

adalah `Run()` yang digunakan untuk menjalankan gerakan zigzag dan tembakan secara terus-menerus selama pertandingan berlangsung. Selain itu, terdapat fungsi `OnScannedBot()` untuk mendeteksi musuh dan langsung menyerang berdasarkan jarak lawan, `OnHitByBullet()` untuk mengubah arah gerak saat terkena tembakan, serta `OnHitWall()` untuk menghindari tabrakan dengan dinding arena. Seluruh proses pada `BotKeempat` dirancang menggunakan strategi greedy by chaos movement, yaitu selalu memilih gerakan yang sulit diprediksi lawan agar bot dapat bertahan lebih lama dan memperoleh score setinggi mungkin. Strategi ini terbukti efektif karena `BotKeempat` berhasil menjadi bot dengan total score tertinggi pada hasil pertandingan.

4.2.1 Struktur Data yang Digunakan

`BotKeempat` menggunakan variabel-variabel sederhana tanpa struktur data kompleks karena strategi greedy by hit probability hanya membutuhkan data dari musuh yang sedang dipindai pada setiap event. Variabel yang digunakan antara lain:

- `double firePower` — menyimpan nilai kekuatan tembakan yang digunakan pada setiap serangan (nilai tetap 3.0 untuk damage maksimal).
- `double bulletSpeed` — kecepatan peluru yang dihitung dari formula $20 - 3 \times \text{firePower}$.
- `double predictedX, predictedY` — koordinat prediksi posisi musuh saat peluru tiba, dihitung berdasarkan posisi, kecepatan, dan arah gerak musuh.
- `double gunAngle` — sudut yang harus diputar senjata agar mengarah ke posisi prediksi musuh.
- `double radarAngle` — sudut yang harus diputar radar agar tetap mengunci musuh dengan teknik oscillation.

Penggunaan variabel lokal pada setiap event dipilih karena data musuh selalu diperbarui setiap kali `OnScannedBot` dipanggil sehingga tidak diperlukan penyimpanan data permanen antar turn.

4.2.2 Fungsi-Fungsi Utama

Berikut merupakan beberapa fungsi utama yang digunakan pada implementasi `BotKeempat`:

16. `Run()` — Fungsi utama yang mengatur warna bot, mengaktifkan mode independen gun dan radar, lalu menjalankan radar sweep 360 derajat secara terus-menerus selama pertandingan berlangsung.

17. OnScannedBot() — Fungsi inti yang dipanggil setiap kali radar mendeteksi musuh. Fungsi ini menghitung prediksi posisi musuh, mengarahkan senjata ke posisi prediksi, dan menembak bila sudut gun sudah cukup akurat.
18. DistanceTo(x, y) — Fungsi bawaan Robocode untuk menghitung jarak antara posisi bot dan posisi musuh. Digunakan untuk menghitung travel time peluru.
19. GunBearingTo(x, y) — Fungsi bawaan Robocode untuk menentukan sudut putaran senjata menuju koordinat target. Digunakan untuk mengarahkan tembakan ke posisi prediksi.
20. RadarBearingTo(x, y) — Fungsi bawaan Robocode untuk menentukan sudut putaran radar menuju koordinat musuh. Digunakan untuk menjaga radar lock pada musuh.

4.2.3 Prosedur Event Handler

BotKeempat menggunakan tiga event handler untuk merespons kondisi yang terjadi selama pertandingan berlangsung.

21. OnScannedBot(ScannedBotEvent e) — Dipanggil setiap kali radar mendeteksi musuh. Bot menghitung prediksi posisi musuh, mengarahkan gun, menembak bila akurat (sudut < 5 derajat), lalu memperbarui posisi radar dengan teknik oscillation.
22. OnHitWall(HitWallEvent e) — Dipanggil saat bot menabrak dinding arena. Bot mundur 30 unit lalu memutar badan 90 derajat untuk keluar dari dinding dan kembali bergerak bebas.
23. OnHitByBullet(HitByBulletEvent e) — Dipanggil saat bot kena tembakan. Bot memutar badan 60 derajat dan maju 80 unit sebagai manuver menghindari dari tembakan berikutnya.

4.3 Pengujian

4.3.1 Pengujian 1: Kondisi Normal pada Arena Standar

Pengujian pertama dilakukan pada arena standar dengan kondisi awal normal. Bot utama BotKeempat diadu melawan BotPertama yang menggunakan strategi greedy by distance. Pada pengujian ini kedua bot memulai pertandingan dengan energi penuh dan posisi awal acak.

Pada awal pertandingan, BotPertama langsung bergerak agresif mendekati BotKeempat. Namun BotKeempat berhasil mendeteksi pergerakan musuh dan menembak ke posisi prediksi sehingga beberapa tembakan mengenai BotPertama sebelum sempat mendekat. Akurasi tembakan prediktif BotKeempat terbukti lebih tinggi dibanding tembakan langsung BotPertama.

[Sisipkan screenshot atau dokumentasi hasil pertandingan pengujian ke-1 di sini]

Parameter	Bot Utama (BotKeempat)	Bot Asisten (BotPertama)
Skor Akhir	[Isi skor]	[Isi skor]
Energi Tersisa	[Isi energi]	[Isi energi]
Jumlah Tembakan	[Isi jumlah]	[Isi jumlah]
Akurasi Tembakan	[Isi %]	[Isi %]
Hasil Pertandingan	[Menang / Kalah]	[Menang / Kalah]

Berdasarkan hasil pengujian, strategi greedy by hit probability bekerja cukup efektif pada arena standar. BotKeempat mampu menekan lawan dengan tembakan akurat sehingga lawan kesulitan mendekati BotKeempat tanpa terkena damage terlebih dahulu.

4.3.2 Pengujian 2: Melawan Bot Agresif

Pada pengujian kedua, BotKeempat diadu melawan BotKeempat yang memiliki pola gerakan chaos dan sangat tidak terprediksi. Arena yang digunakan masih arena standar, namun pertandingan berlangsung lebih dinamis karena BotKeempat terus bergerak secara acak.

BotKeempat tetap menggunakan strategi radar lock dan prediksi linear. Meskipun beberapa tembakan meleset akibat gerakan chaos BotKeempat, BotKeempat tetap mampu mengakumulasi damage karena tembakan dengan power 3.0 menghasilkan damage tinggi setiap kali mengenai target.

[Sisipkan screenshot atau dokumentasi hasil pertandingan pengujian ke-2 di sini]

Parameter	Bot Utama (BotKeempat)	Bot Asisten (BotKeempat)
Skor Akhir	[Isi skor]	[Isi skor]
Energi Tersisa	[Isi energi]	[Isi energi]
Jumlah Tembakan	[Isi jumlah]	[Isi jumlah]
Akurasi Tembakan	[Isi %]	[Isi %]

Parameter	Bot Utama (BotKeempat)	Bot Asisten (BotKeempat)
Hasil Pertandingan	[Menang / Kalah]	[Menang / Kalah]

Hasil pengujian menunjukkan bahwa BotKeempat masih mampu memberikan perlawanan yang baik meskipun menghadapi bot dengan gerakan chaos. Namun akurasi tembakan menurun dibanding pengujian pertama karena gerakan acak BotKeempat menyulitkan prediksi linear.

4.3.3 Pengujian 3: Melawan Bot Defensif

Pada pengujian ketiga, BotKeempat dihadapkan melawan BotKetiga yang menggunakan strategi wall hugger dengan pergerakan terus-menerus di sepanjang dinding. Skenario ini menguji kemampuan BotKeempat dalam memprediksi posisi musuh yang bergerak dengan pola linear namun terus-menerus.

Pergerakan linear BotKetiga justru memudahkan prediksi BotKeempat karena kecepatan dan arah gerak musuh relatif konsisten. Radar lock BotKeempat berhasil mempertahankan target sepanjang pertandingan dan akurasi tembakan meningkat signifikan pada skenario ini.

[Sisipkan screenshot atau dokumentasi hasil pertandingan pengujian ke-3 di sini]

Parameter	Bot Utama (BotKeempat)	Bot Asisten (BotKetiga)
Skor Akhir	[Isi skor]	[Isi skor]
Energi Tersisa	[Isi energi]	[Isi energi]
Jumlah Tembakan	[Isi jumlah]	[Isi jumlah]
Akurasi Tembakan	[Isi %]	[Isi %]
Hasil Pertandingan	[Menang / Kalah]	[Menang / Kalah]

Pada pengujian ini terlihat bahwa strategi greedy by hit probability bekerja paling optimal ketika menghadapi musuh dengan pola gerak yang konsisten. Gerakan linear BotKetiga membuat prediksi BotKeempat sangat akurat sehingga sebagian besar tembakan mengenai target.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, strategi Greedy by Hit Probability yang digunakan pada BotKeempat terbukti cukup efektif dalam pertandingan 1

vs 1. Bot mampu menghasilkan akurasi tembakan yang tinggi berkat mekanisme prediksi posisi musuh dan radar lock yang bekerja secara berkelanjutan.

Strategi ini memberikan keuntungan utama berupa akurasi tembakan yang jauh lebih tinggi dibanding bot yang hanya menembak ke posisi saat ini. Tembakan dengan power 3.0 yang akurat menghasilkan damage besar setiap kali mengenai target. Selain itu radar oscillation memastikan BotKeempat tidak kehilangan target meskipun musuh bergerak ke berbagai arah.

Namun, strategi ini memiliki kelemahan saat menghadapi musuh dengan gerakan sangat acak seperti BotKeempat, karena prediksi linear tidak dapat mengantisipasi perubahan arah mendadak. Pada kondisi tersebut, akurasi tembakan BotKeempat menurun karena posisi prediksi sering meleset dari posisi aktual musuh.

Secara keseluruhan, strategi Greedy by Chaos Movement cocok digunakan pada BotKeempat karena menghasilkan pergerakan yang sulit diprediksi, kemampuan bertahan tinggi, dan performa paling stabil selama pertandingan.

BAB 5: KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, algoritma greedy dapat diterapkan dengan baik pada permainan Robocode Tank Royale. Keempat bot berhasil mengimplementasikan heuristic greedy yang berbeda sesuai dengan tujuan strateginya masing-masing.

Empat strategi greedy yang berhasil dikembangkan dalam tugas besar ini yaitu:

- Greedy by Distance pada BotPertama.
- Greedy by Hit Probability pada BotKeempat.
- Greedy by Safe Movement pada BotKetiga.
- Greedy by Chaos Movement pada BotKeempat.

Dari seluruh strategi yang diuji, strategi Greedy by Chaos Movement yang diterapkan pada BotKeempat dipilih sebagai strategi utama karena berhasil memperoleh total score tertinggi dan memenangkan pertandingan terbanyak. Strategi ini mampu memprediksi posisi musuh saat peluru tiba dan menembak ke posisi masa depan tersebut sehingga peluang mengenai target menjadi lebih besar dibanding strategi lainnya.

Melalui tugas besar ini dapat dipahami bahwa algoritma greedy sangat cocok digunakan pada sistem permainan real-time karena memiliki kompleksitas rendah, proses pengambilan keputusan yang cepat, dan implementasi yang relatif sederhana namun tetap efektif bila didukung heuristic yang tepat.

5.2 Saran

Beberapa saran pengembangan yang dapat dilakukan pada penelitian dan implementasi berikutnya adalah sebagai berikut:

- Mengembangkan sistem prediksi pergerakan musuh yang lebih kompleks dengan mempertimbangkan akselerasi dan pola historis gerakan musuh, tidak hanya kecepatan dan arah saat ini.
- Mengombinasikan algoritma greedy dengan algoritma lain seperti minimax atau reinforcement learning untuk menghasilkan strategi yang lebih optimal.
- Menambahkan mekanisme penghindaran tembakan yang lebih proaktif, misalnya dengan mendeteksi penurunan energi musuh sebagai indikator bahwa musuh baru saja menembak.

- Menambahkan mekanisme penghindaran dinding yang lebih baik agar bot tidak kehilangan posisi saat fokus mengunci target.
- Mengembangkan strategi kerja sama antar bot pada mode multiplayer sehingga bot dapat saling mendukung dalam pertandingan tim.

LAMPIRAN

Lampiran A: Tautan Repository GitHub

Repository kode sumber bot dapat diakses melalui tautan berikut:

GitHub: [Masukkan URL repository GitHub di sini]

Lampiran B: Tautan Video Demo (Opsional)

Video demonstrasi bot (jika ada) dapat diakses melalui tautan berikut:

Video: [Masukkan URL video di sini, atau hapus bagian ini jika tidak ada video]

DAFTAR PUSTAKA

- [1] Robocode Tank Royale. (2024). Official Documentation. Tersedia di: <https://robocode-dev.github.io/tank-royale/>
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- [3] [Penulis]. ([Tahun]). [Judul referensi tentang algoritma greedy]. [Sumber]. Tersedia di: [URL]
- [4] [Tambahkan referensi lain sesuai kebutuhan. Gunakan format sitasi yang konsisten: IEEE atau APA]

HASIL PERTANDINGAN 10 RONDE

Berdasarkan hasil pertandingan 10 ronde, BotKeempat berhasil memperoleh total score tertinggi sebesar 1527 poin. BotKeempat juga memperoleh survival score tertinggi sebesar 600 dan bullet damage sebesar 702. Selain itu, BotKeempat berhasil memperoleh 4 kemenangan (1st place), menjadikannya bot paling efektif dibandingkan bot lainnya. Keunggulan utama BotKeempat berasal dari strategi Greedy by Chaos Movement yang membuat pola gerakan bot sangat sulit diprediksi lawan. Gerakan zigzag acak dan perubahan arah mendadak menyebabkan lawan sering gagal mengenai target. Strategi ini juga memungkinkan BotKeempat bertahan hidup lebih lama sehingga memperoleh survival score yang tinggi.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	BotKeempat 4.0	1527	600	120	702	88	17	0	4	0	0
2	BotKetiga 3.0	676	350	0	282	15	29	0	0	3	1
3	BotPertama 1.0	226	50	0	112	0	64	0	0	0	2
4	BotKedua 2.0	202	200	0	0	0	2	0	0	1	1